

Matching Regular-Typed Pattern Languages: Quadratic-Time Algorithms

Yuya Uezato

CyberAgent, Inc., Japan

CPM 2026



Our Problem: Membership Problem



Given an input text W and a regular-typed pattern expression $\mathcal{R} = (P, C)$, decide whether W belongs to the language of $\mathcal{R}$:

$$\overbrace{W}^{\text{input text}} \in L_{\underbrace{C}_{\text{regular type}}}(\overbrace{P}^{\text{pattern}}) ?$$

Our Problem: Membership Problem



Given an input text W and a regular-typed pattern expression $\mathcal{R} = (P, C)$, decide whether W belongs to the language of $\mathcal{R}$:

$$\overbrace{W}^{\text{input text}} \in L_{\underbrace{C}_{\text{regular type}}}(\overbrace{P}^{\text{pattern}}) ?$$

Main object: a regular-typed pattern expression $\mathcal{R} = (P, C)$
and its language $L_C(P)$.

Our Problem: Membership Problem



Given an input text W and a regular-typed pattern expression $\mathcal{R} = (P, C)$, decide whether W belongs to the language of $\mathcal{R}$:

$$\overbrace{W}^{\text{input text}} \in L_{\underbrace{C}_{\text{regular type}}}(\overbrace{P}^{\text{pattern}}) ?$$

Main object: a regular-typed pattern expression $\mathcal{R} = (P, C)$
and its language $L_C(P)$.

Main Result

For regular-typed patterns with a single repeated variable, this problem can be solved in

$$O(|W|^2 \text{ poly}(|\mathcal{R}|)) \text{ time.}$$

Preliminaries: Pattern Expressions *without Types*

A pattern expression P is a string over alphabet symbols and variables.

Let $\Sigma = \{0, 1, \#\}$ be an alphabet and let α be a variable.

$$\text{(Example) } P = \alpha \# \# \alpha .$$

Preliminaries: Pattern Expressions *without Types*

A pattern expression P is a string over alphabet symbols and variables.

Let $\Sigma = \{0, 1, \#\}$ be an alphabet and let α be a variable.

$$\text{(Example) } P = \alpha \# \# \alpha.$$

The language of P is obtained by assigning an arbitrary string $w \in \Sigma^*$ to α :

$$L(P) := \{ w_\alpha \# \# w_\alpha : w_\alpha \in \Sigma^* \}.$$

Preliminaries: Pattern Expressions *without Types*

A pattern expression P is a string over alphabet symbols and variables.

Let $\Sigma = \{0, 1, \#\}$ be an alphabet and let α be a variable.

$$\text{(Example)} \quad P = \alpha \# \# \alpha.$$

The language of P is obtained by assigning an arbitrary string $w \in \Sigma^*$ to α :

$$L(P) := \{ w_\alpha \# \# w_\alpha : w_\alpha \in \Sigma^* \}.$$

Example

$$010 \# \# 010 \in L(P), \quad \text{by } \alpha \mapsto 010,$$

Preliminaries: Pattern Expressions *without Types*

A pattern expression P is a string over alphabet symbols and variables.

Let $\Sigma = \{0, 1, \#\}$ be an alphabet and let α be a variable.

$$\text{(Example)} \quad P = \alpha \#\#\alpha.$$

The language of P is obtained by assigning an arbitrary string $w \in \Sigma^*$ to α :

$$L(P) := \{ w_\alpha \#\# w_\alpha : w_\alpha \in \Sigma^* \}.$$

Example

$$\begin{array}{ll} 010 \#\# 010 \in L(P), & \text{by } \alpha \mapsto 010, \\ 010 \#\# 1011 \notin L(P), & \text{because } 010 \neq 1011 \end{array}$$

Preliminaries: Pattern Expressions *without Types*

A pattern expression P is a string over alphabet symbols and variables.

Let $\Sigma = \{0, 1, \#\}$ be an alphabet and let α be a variable.

$$\text{(Example)} \quad P = \alpha \#\#\alpha.$$

The language of P is obtained by assigning an arbitrary string $w \in \Sigma^*$ to α :

$$L(P) := \{ w_\alpha \#\# w_\alpha : w_\alpha \in \Sigma^* \}.$$

Example

$010 \#\# 010 \in L(P),$ by $\alpha \mapsto 010,$
 $010 \#\# 1011 \notin L(P),$ because $010 \neq 1011$
 $0 \#\# 1 \#\# 0 \#\# 1 \in L(P),$ by $\alpha \mapsto 0 \#\# 1.$

Main Object: Pattern Expressions with *Regular Types*

For $P = \alpha \#\#\alpha$ and $\Sigma = \{0, 1, \#\}$, its language $L(P)$ is

$$\overbrace{0 \#\# 1}^{\alpha} \#\# \overbrace{0 \#\# 1}^{\alpha} \in L(P) = \{w_{\alpha} \#\# w_{\alpha} : w_{\alpha} \in \Sigma^*\}$$

Main Object: Pattern Expressions with *Regular Types*

For $P = \alpha \#\#\alpha$ and $\Sigma = \{0, 1, \#\}$, its language $L(P)$ is

$$\overbrace{0\#\#1}^{\alpha} \#\# \overbrace{0\#\#1}^{\alpha} \in L(P) = \{w_{\alpha}\#\#w_{\alpha} : w_{\alpha} \in \Sigma^*\}$$

Role of Types: *Restrict* the **domain** of variables.

In particular, a *regular type* restricts variables by regular languages:

$$\text{(Example): } C(\alpha) = (\Sigma \setminus \{\#\})^* = \{0, 1\}^*.$$

Main Object: Pattern Expressions with *Regular Types*

For $P = \alpha \#\#\alpha$ and $\Sigma = \{0, 1, \#\}$, its language $L(P)$ is

$$\overbrace{0\#\#1}^{\alpha} \#\# \overbrace{0\#\#1}^{\alpha} \in L(P) = \{w_{\alpha}\#\#w_{\alpha} : w_{\alpha} \in \Sigma^*\}$$

Role of Types: *Restrict* the **domain** of variables.

In particular, a *regular type* restricts variables by regular languages:

$$\text{(Example): } C(\alpha) = (\Sigma \setminus \{\#\})^* = \{0, 1\}^*.$$

The regular-typed pattern language $L_C(P)$ is defined as follows:

$$L_C(P) = \{w_{\alpha}\#\#w_{\alpha} : w_{\alpha} \in C(\alpha)\} = \{w_{\alpha}\#\#w_{\alpha} : w_{\alpha} \in \{0, 1\}^*\}.$$

Main Object: Pattern Expressions with *Regular Types*

For $P = \alpha \#\#\alpha$ and $\Sigma = \{0, 1, \#\}$, its language $L(P)$ is

$$\overbrace{0\#\#1}^{\alpha} \#\# \overbrace{0\#\#1}^{\alpha} \in L(P) = \{w_{\alpha}\#\#w_{\alpha} : w_{\alpha} \in \Sigma^*\}$$

Role of Types: *Restrict* the **domain** of variables.

In particular, a *regular type* restricts variables by regular languages:

$$\text{(Example): } C(\alpha) = (\Sigma \setminus \{\#\})^* = \{0, 1\}^*.$$

The regular-typed pattern language $L_C(P)$ is defined as follows:

$$L_C(P) = \{w_{\alpha}\#\#w_{\alpha} : w_{\alpha} \in C(\alpha)\} = \{w_{\alpha}\#\#w_{\alpha} : w_{\alpha} \in \{0, 1\}^*\}.$$

Example

$$010\#\#010 \in L_C(P), \quad 0\#\#1\#\#0\#\#1 \notin L_C(P).$$

Formal Definition of Our Problem: Single Repeated Variable

We focus on regular-typed patterns with a *single repeated variable* β as follows:

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K \quad (K \geq 3)$$

where $\alpha_i \neq \alpha_j$ for $i \neq j$.

Example

$$P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3, \quad P_4 = \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

Formal Definition of Our Problem: Single Repeated Variable

We focus on regular-typed patterns with a *single repeated variable* β as follows:

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K \quad (K \geq 3)$$

where $\alpha_i \neq \alpha_j$ for $i \neq j$.

Example

$$P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3, \quad P_4 = \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

Our Main Problem (in This Talk)

Fixed Param.: Σ ; $C(\alpha_i)$ is a deterministic finite automaton (DFA) $\mathcal{A}_i$ and $C(\beta)$ is a DFA $\mathcal{B}$.

Input: a string W

Formal Definition of Our Problem: Single Repeated Variable

We focus on regular-typed patterns with a *single repeated variable* β as follows:

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K \quad (K \geq 3)$$

where $\alpha_i \neq \alpha_j$ for $i \neq j$.

Example

$$P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3, \quad P_4 = \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

Our Main Problem (in This Talk)

Fixed Param.: Σ ; $C(\alpha_i)$ is a deterministic finite automaton (DFA) $\mathcal{A}_i$ and $C(\beta)$ is a DFA $\mathcal{B}$.

Input: a string W

Task: Find a decomposition of W of the following form:

$$(K = 3) \quad W = \boxed{w_1} \boxed{u} \boxed{w_2} \boxed{u} \boxed{w_3}$$

$\text{---} \in \mathcal{A}_1 \text{---} \quad \text{---} \in \mathcal{B} \text{---} \quad \text{---} \in \mathcal{A}_2 \text{---} \quad \quad \text{---} \in \mathcal{A}_3 \text{---}$

Why Study This Form $\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$?

Motivation: Extending Related Work

Work	Generality	Type	Time Complexity
Fernau et al. [STACS'15, TOCT'20]	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	NO	$O(W ^2 \text{ poly}(P))$

Why Study This Form $\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$?

Motivation: Extending Related Work

Work	Generality	Type	Time Complexity
Fernau et al. [STACS'15, TOCT'20]	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	NO	$O(W ^2 \text{ poly}(P))$
Nogami–Terauchi [MFCS'25]	Only $K = 3$: $\alpha_1 \beta \alpha_2 \beta \alpha_3$	YES	$O(W ^2 \text{ poly}(P + C))$

Why Study This Form $\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$?

Motivation: Extending Related Work

Work	Generality	Type	Time Complexity
Fernau et al. [STACS'15, TOCT'20]	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	NO	$O(W ^2 \text{ poly}(P))$
Nogami–Terauchi [MFCS'25]	Only $K = 3$: $\alpha_1 \beta \alpha_2 \beta \alpha_3$	YES	$O(W ^2 \text{ poly}(P + C))$
This Work	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	YES	$O(W ^2 \text{ poly}(P + C))$

Why Study This Form $\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$?

Motivation: Extending Related Work

Work	Generality	Type	Time Complexity
Fernau et al. [STACS'15, TOCT'20]	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	NO	$O(W ^2 \text{ poly}(P))$
Nogami–Terauchi [MFCS'25]	Only $K = 3$: $\alpha_1 \beta \alpha_2 \beta \alpha_3$	YES	$O(W ^2 \text{ poly}(P + C))$
This Work	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	YES	$O(W ^2 \text{ poly}(P + C))$

Practical Motivation: A Real-World Backreference Fragment

P_K can be written as an *actual* regular expression using **backreferences**:

$$\text{For } K = 3, \quad \alpha_1 \beta \alpha_2 \beta \alpha_3 \quad \Leftrightarrow \quad A_1 (B)_1 A_2 \setminus 1 A_3,$$

where $A_i = C(\alpha_i)$ and $B = C(\beta)$ are regular expressions.

Why Study This Form $\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$?

Motivation: Extending Related Work

Work	Generality	Type	Time Complexity
Fernau et al. [STACS'15, TOCT'20]	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	NO	$O(W ^2 \text{ poly}(P))$
Nogami–Terauchi [MFCS'25]	Only $K = 3$: $\alpha_1 \beta \alpha_2 \beta \alpha_3$	YES	$O(W ^2 \text{ poly}(P + C))$
This Work	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	YES	$O(W ^2 \text{ poly}(P + C))$

Practical Motivation: A Real-World Backreference Fragment

P_K can be written as an *actual* regular expression using **backreferences**:

$$\text{For } K = 3, \quad \alpha_1 \beta \alpha_2 \beta \alpha_3 \quad \Leftrightarrow \quad A_1 (B)_1 A_2 \setminus 1 A_3,$$

where $A_i = C(\alpha_i)$ and $B = C(\beta)$ are regular expressions.

🌟 Developing **simple** and **implementable** algorithms for **real-world** regular expressions with *backreferences* is important.

Talk Overview

So far

- We introduced regular-typed pattern expressions. ✓
- We saw examples. ✓
- We fixed the target form: $P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$. ✓

Talk Overview

So far

- We introduced regular-typed pattern expressions. ✓
- We saw examples. ✓
- We fixed the target form: $P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$. ✓

Next: Explaining Our Algorithm for $K = 3$ and $K = 4$

We will see an $O(|W|^2)$ -time algorithm for the smallest nontrivial case:

$$K = 3 : \quad \alpha_1 \beta \alpha_2 \beta \alpha_3,$$

then extend it to the case (if we have enough time):

$$K = 4 : \quad \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

Talk Overview

So far

- We introduced regular-typed pattern expressions. ✓
- We saw examples. ✓
- We fixed the target form: $P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$. ✓

Next: Explaining Our Algorithm for $K = 3$ and $K = 4$

We will see an $O(|W|^2)$ -time algorithm for the smallest nontrivial case:

$$K = 3 : \quad \alpha_1 \beta \alpha_2 \beta \alpha_3,$$

then extend it to the case (if we have enough time):

$$K = 4 : \quad \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

$K = 3$ is the key case; the general $K \geq 4$ is a simple extension.

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,
 our **task** is as follows:

Find a decomposition $W = ?$

$\vdash \in \mathcal{A}_1 \dashv$	$\vdash \in \mathcal{B} \dashv$	$\vdash \in \mathcal{A}_2 \dashv$	$\vdash \in \mathcal{B} \dashv$	$\vdash \in \mathcal{A}_3 \dashv$
w_1	u	w_2	u	w_3

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,
 our **task** is as follows:

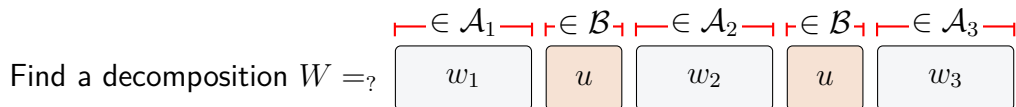
Find a decomposition $W = ?$

$\vdash \in \mathcal{A}_1 \dashv$	$\vdash \in \mathcal{B} \dashv$	$\vdash \in \mathcal{A}_2 \dashv$	$\vdash \in \mathcal{B} \dashv$	$\vdash \in \mathcal{A}_3 \dashv$
w_1	u	w_2	u	w_3

Preprocessing: Two Data Structures

- 1 Build a data structure for constant-time DFA queries in $O(|W|^2)$ time.

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,
 our **task** is as follows:



Preprocessing: Two Data Structures

- ① Build a data structure for constant-time DFA queries in $O(|W|^2)$ time.
- ② Build sorted occurrence lists $\text{Subs}_{\text{occ}}(W)$ in $O(|W|^2)$ time.

Build Data Structures for DFA Queries

Recap Notation: Let $\mathcal{A}$ be a **deterministic** finite automaton (DFA) and $q \in Q_{\mathcal{A}}$ be a state.

Build Data Structures for DFA Queries

Recap Notation: Let $\mathcal{A}$ be a **deterministic** finite automaton (DFA) and $q \in Q_{\mathcal{A}}$ be a state. We write $q \xrightarrow{\sigma} q'$ to denote that reading the *symbol* σ from q leads to q' .

Build Data Structures for DFA Queries

Recap Notation: Let $\mathcal{A}$ be a **deterministic** finite automaton (DFA) and $q \in Q_{\mathcal{A}}$ be a state.

We write $q \xrightarrow{\sigma} q'$ to denote that reading the *symbol* σ from q leads to q' .

For a string $u \in \Sigma^*$, we write $q \xRightarrow{u} q'$ to denote that we reach q' by reading u from q .

Build Data Structures for DFA Queries

Recap Notation: Let $\mathcal{A}$ be a **deterministic** finite automaton (DFA) and $q \in Q_{\mathcal{A}}$ be a state.

We write $q \xrightarrow{\sigma} q'$ to denote that reading the *symbol* σ from q leads to q' .

For a string $u \in \Sigma^*$, we write $q \xRightarrow{u} q'$ to denote that we reach q' by reading u from q .

Equivalently, we write $\mathcal{A}(q, u)$ for such q' .

Build Data Structures for DFA Queries

Recap Notation: Let $\mathcal{A}$ be a **deterministic** finite automaton (DFA) and $q \in Q_{\mathcal{A}}$ be a state. We write $q \xrightarrow{\sigma} q'$ to denote that reading the *symbol* σ from q leads to q' .

For a string $u \in \Sigma^*$, we write $q \xRightarrow{u} q'$ to denote that we reach q' by reading u from q .

Equivalently, we write $\mathcal{A}(q, u)$ for such q' .

Preprocessing. For every state $q \in Q_{\mathcal{A}}$ and index i, j of W ,

we build the table

$$\Delta_{\mathcal{A}}[q][i][j] := \mathcal{A}(q, W[i..j]).$$

in $O(|W|^2)$ time. We treat each substring by its positions $[i, j)$.

Build Data Structures for DFA Queries

Recap Notation: Let $\mathcal{A}$ be a **deterministic** finite automaton (DFA) and $q \in Q_{\mathcal{A}}$ be a state. We write $q \xrightarrow{\sigma} q'$ to denote that reading the *symbol* σ from q leads to q' .

For a string $u \in \Sigma^*$, we write $q \xRightarrow{u} q'$ to denote that we reach q' by reading u from q .

Equivalently, we write $\mathcal{A}(q, u)$ for such q' .

Preprocessing. For every state $q \in Q_{\mathcal{A}}$ and index i, j of W , we build the table

$$\Delta_{\mathcal{A}}[q][i][j] := \mathcal{A}(q, W[i..j]).$$

in $O(|W|^2)$ time. We treat each substring by its positions $[i, j)$.

Application: We can test whether $\mathcal{A}$ accepts the substring $W[i..j)$ in $O(1)$ time by checking

$$\Delta_{\mathcal{A}}[q^{\text{init}}][i][j] \in \text{Final}_{\mathcal{A}} ?$$

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,
our task is as follows:

Find a decomposition $W = ?$

w_1	u	w_2	u	w_3
-------	-----	-------	-----	-------

$\vdash \in \mathcal{A}_1 \dashv$
 $\vdash \in \mathcal{B} \dashv$
 $\vdash \in \mathcal{A}_2 \dashv$
 $\vdash \in \mathcal{B} \dashv$
 $\vdash \in \mathcal{A}_3 \dashv$

Preprocessing: Two Data Structures

- ① Build a data structure for constant-time DFA queries in $O(|W|^2)$ time. ✓
- ② Build sorted occurrence lists $\text{Subs}_{\text{occ}}(W)$ in $O(|W|^2)$ time. 🙄

Sub_{Socc}(W): Sorted Occurrence Lists for Substrings

Let u be a substring of W , we define the sorted occurrence list of u in W as follows:

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \left[\cdots \right]_{p_1} \left[u_1 \right] \left[\cdots \right]_{p_2} \left[u_2 \right] \left[\cdots \right]_{p_3} \left[u_3 \right] \left[\cdots \right]_{p_r} \left[u_r \right] \left[\cdots \right]$$

Sub_{Socc}(W): Sorted Occurrence Lists for Substrings

Let u be a substring of W , we define the sorted occurrence list of u in W as follows:

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \left[\cdots \right]_{p_1} \left[u_1 \right] \left[\cdots \right]_{p_2} \left[u_2 \right] \left[\cdots \right]_{p_3} \left[u_3 \right] \left[\cdots \right]_{p_r} \left[u_r \right] \left[\cdots \right]$$

Example

$$W = \begin{array}{cccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ a & b & a & a & b & a & a & b & b & a \end{array}$$

Sub_{Socc}(W): Sorted Occurrence Lists for Substrings

Let u be a substring of W , we define the sorted occurrence list of u in W as follows:

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \left[\cdots \right]_{p_1}^{u_1} \left[\cdots \right]_{p_2}^{u_2} \left[\cdots \right]_{p_3}^{u_3} \left[\cdots \right]_{p_r}^{u_r} \left[\cdots \right]$$

Example

$$W = \begin{array}{cccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ a & \underline{b} & a & a & \underline{b} & a & a & b & \underline{b} & a \end{array}$$

$$\text{Socc}(\underline{ba}) = [2, 5, 9]$$

Sub_{Socc}(W): Sorted Occurrence Lists for Substrings

Let u be a substring of W , we define the sorted occurrence list of u in W as follows:

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \left[\begin{array}{c} \cdots \\ \boxed{u_1} \\ p_1 \end{array} \right] \left[\begin{array}{c} \cdots \\ \boxed{u_2} \\ p_2 \end{array} \right] \left[\begin{array}{c} \cdots \\ \boxed{u_3} \\ p_3 \end{array} \right] \left[\begin{array}{c} \cdots \\ \boxed{u_r} \\ p_r \end{array} \right] \cdots$$

Example

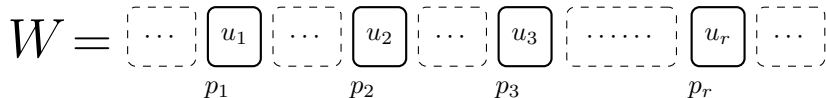
$$W = \begin{array}{cccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ a & b & a & a & b & a & a & b & b & a \end{array}$$

$$\text{Socc}(ba) = [2, 5, 9] \quad \text{Socc}(abaab) = [1, 4]$$

Sub_{occ}(W): Sorted Occurrence Lists for Substrings

Let u be a substring of W , we define the sorted occurrence list of u in W as follows:

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$



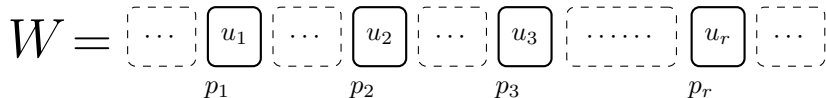
Our data structure

$$\text{Subs}_{\text{occ}}(W) = \{ \langle u, \text{Socc}(u) \rangle : u \text{ is a nonempty substring of } W \}.$$

Sub_{occ}(W): Sorted Occurrence Lists for Substrings

Let u be a substring of W , we define the sorted occurrence list of u in W as follows:

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$



Our data structure

$$\text{Subs}_{\text{occ}}(W) = \{ \langle u, \text{Socc}(u) \rangle : u \text{ is a nonempty substring of } W \}.$$

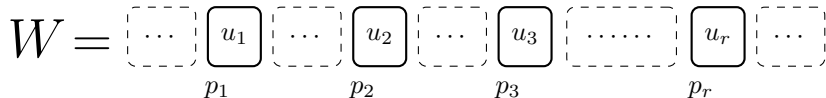
Properties

Fact 1: $\sum_u |\text{Socc}(u)| = \Theta(|W|^2).$

Sub_{occ}(W): Sorted Occurrence Lists for Substrings

Let u be a substring of W , we define the sorted occurrence list of u in W as follows:

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$



Our data structure

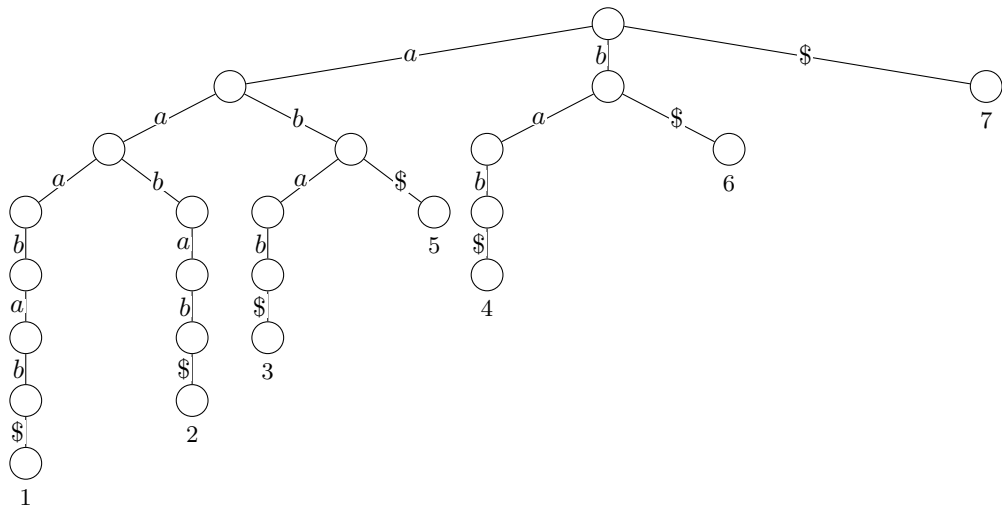
$$\text{Subs}_{\text{occ}}(W) = \{ \langle u, \text{Socc}(u) \rangle : u \text{ is a nonempty substring of } W \}.$$

Properties

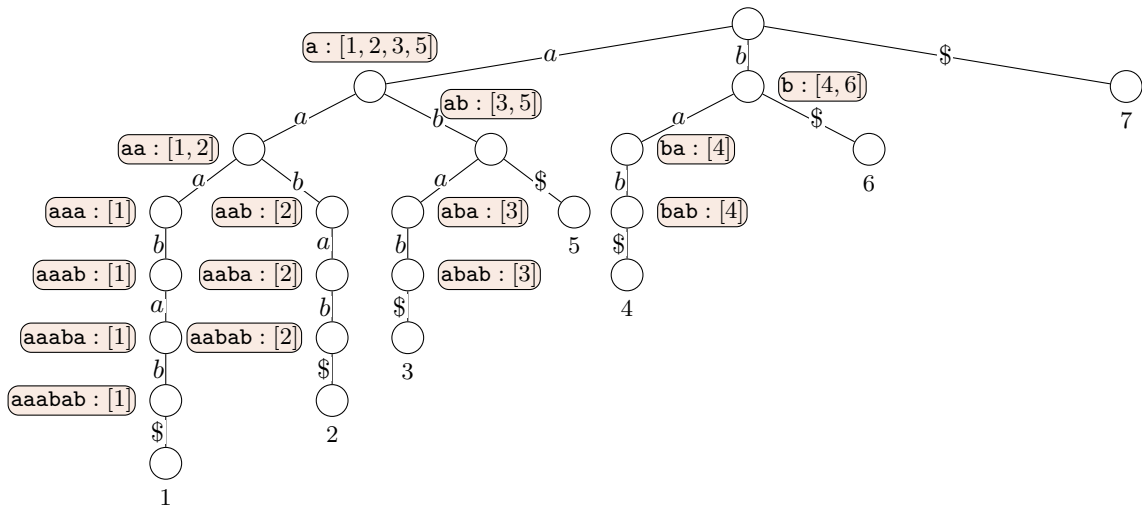
Fact 1: $\sum_u |\text{Socc}(u)| = \Theta(|W|^2)$.

Fact 2: $\text{Subs}_{\text{occ}}(W)$ can be built in $O(|W|^2)$ time by using **your favorite algorithm**.

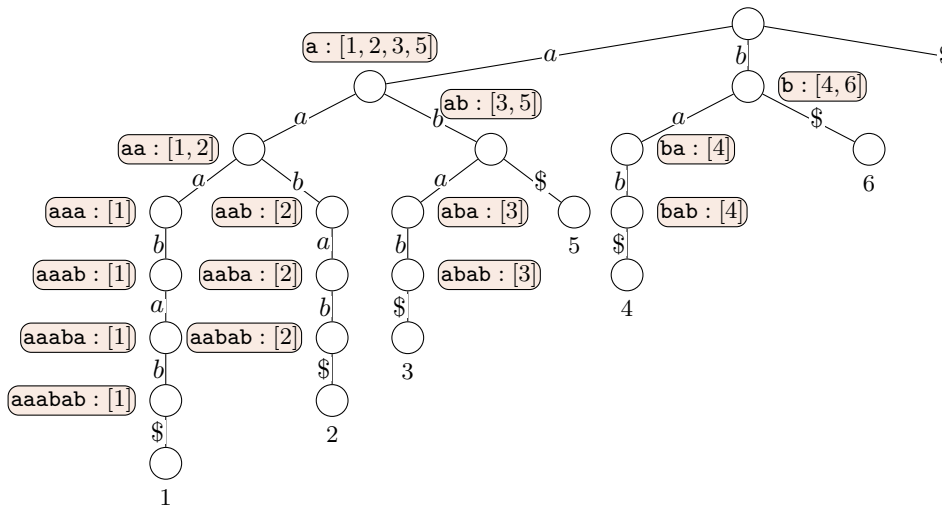
Example: Decorated Suffix Trie of $W = \text{aaabab}$ and $W\$ = \text{aaabab\$}$



Example: Decorated Suffix Trie of $W = \text{aaabab}$ and $W\$ = \text{aaabab\$}$



Example: Decorated Suffix Trie of $W = \text{aaabab}$ and $W\$ = \text{aaabab\$}$



Claim

$\text{Subs}_{\text{occ}}(W)$

can be built in

$O(|W|^2)$ time

by using
postorder
sorted-array
merging.

Note: Σ is fixed.

Algorithm

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,

Find a decomposition $W = ?$

w_1	u	w_2	u	w_3
-------	-----	-------	-----	-------

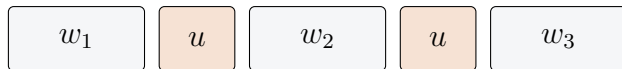
$\vdash \in \mathcal{A}_1 \vdash \vdash \in \mathcal{B} \vdash \vdash \in \mathcal{A}_2 \vdash \vdash \in \mathcal{B} \vdash \vdash \in \mathcal{A}_3 \vdash$

Algorithm

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,

$\text{---} \in \mathcal{A}_1 \text{---} \text{---} \in \mathcal{B} \text{---} \text{---} \in \mathcal{A}_2 \text{---} \text{---} \in \mathcal{B} \text{---} \text{---} \in \mathcal{A}_3 \text{---}$

Find a decomposition $W = ?$



Main Procedure

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

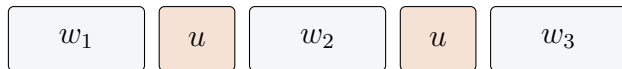
▷ $S_u = [p_1, p_2, \dots, p_r]$ is the sorted occurrence list of u

Algorithm

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,

$\text{---} \in \mathcal{A}_1 \text{---} \text{---} \in \mathcal{B} \text{---} \text{---} \in \mathcal{A}_2 \text{---} \text{---} \in \mathcal{B} \text{---} \text{---} \in \mathcal{A}_3 \text{---}$

Find a decomposition $W = ?$



Main Procedure

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ $S_u = [p_1, p_2, \dots, p_r]$ is the sorted occurrence list of u

if $u \notin L(\mathcal{B})$ **then** ▷ $O(1)$ time check by using the data structure for $\mathcal{B}$

└ **continue**

Algorithm

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,

Find a decomposition $W = ?$

w_1	u	w_2	u	w_3
-------	-----	-------	-----	-------

$\text{---} \in \mathcal{A}_1 \text{---} \text{---} \in \mathcal{B} \text{---} \text{---} \in \mathcal{A}_2 \text{---} \text{---} \in \mathcal{B} \text{---} \text{---} \in \mathcal{A}_3 \text{---}$

Main Procedure

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

$\triangleright S_u = [p_1, p_2, \dots, p_r]$ is the sorted occurrence list of u

if $u \notin L(\mathcal{B})$ **then** $\triangleright O(1)$ time check by using the data structure for $\mathcal{B}$

continue

if $\text{Decompose}(u, S_u)$ **then**

$\triangleright$ check if there is a decomposition $W = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

return ACCEPT

Algorithm

For $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$ with DFAs $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ and $\mathcal{B}$,

Find a decomposition $W = ?$

w_1	u	w_2	u	w_3
-------	-----	-------	-----	-------

$\text{---} \in \mathcal{A}_1 \text{---} \text{---} \in \mathcal{B} \text{---} \text{---} \in \mathcal{A}_2 \text{---} \text{---} \in \mathcal{B} \text{---} \text{---} \in \mathcal{A}_3 \text{---}$

Main Procedure

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

 ▷ $S_u = [p_1, p_2, \dots, p_r]$ is the sorted occurrence list of u

if $u \notin L(\mathcal{B})$ **then** ▷ $O(1)$ time check by using the data structure for $\mathcal{B}$

continue

if $\text{Decompose}(u, S_u)$ **then**

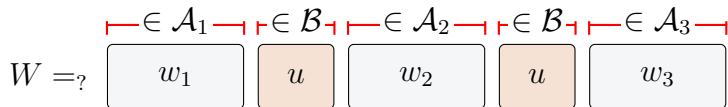
 ▷ check if there is a decomposition $W = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

return ACCEPT

return REJECT

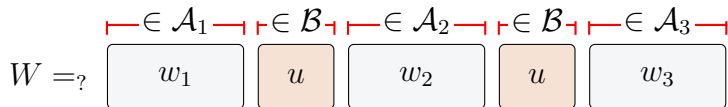
Decompose(u, S_u)

Task: Given a *fixed* candidate substring u ,
our task is to decide if there is a decomposition of the following form in $O(|S_u|)$ time.



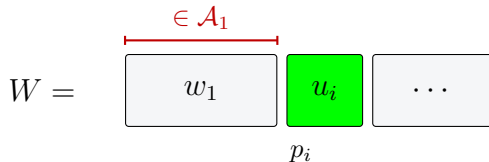
Decompose(u, S_u)

Task: Given a *fixed* candidate substring u ,
our task is to decide if there is a decomposition of the following form in $O(|S_u|)$ time.



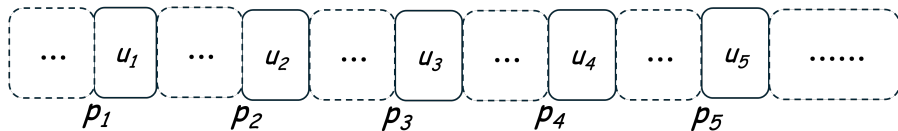
Our Approach.

First, we mark each occurrence u_i starting at $p_i \in S_u$
that satisfies $W[..p_i] \in L(\mathcal{A}_1)$, in total $O(|S_u|)$ time:



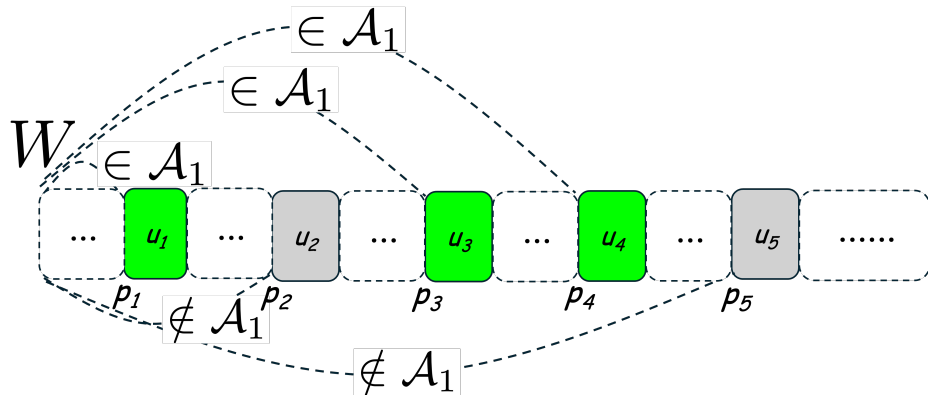
Step 1: Marking Candidates by $\mathcal{A}_1$

W



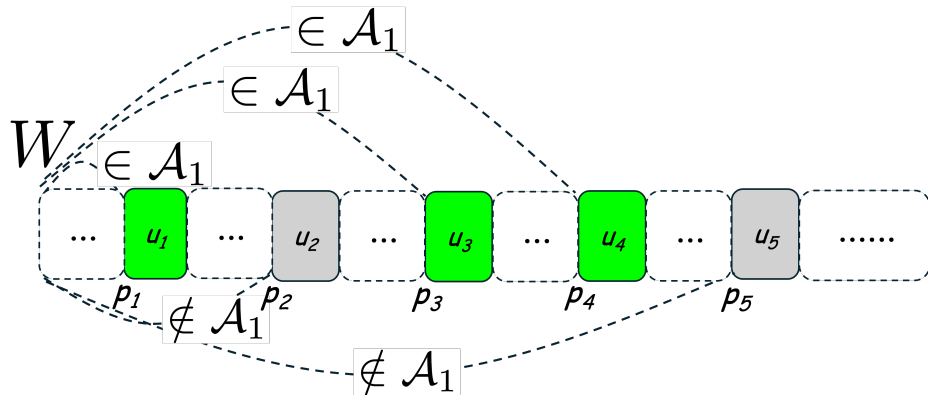
For each $p_i \in S_u$,

Step 1: Marking Candidates by $\mathcal{A}_1$



For each $p_i \in S_u$, we just test if $W[..p_i) \in L(\mathcal{A}_1)$.

Step 1: Marking Candidates by $\mathcal{A}_1$



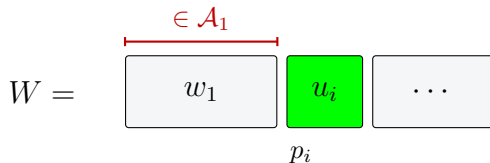
For each $p_i \in S_u$, we just test if $W[..p_i) \in L(\mathcal{A}_1)$.

Each test can be done in $O(1)$ time using the query table of $\mathcal{A}_1$.

Thus, this marking step can be done in $O(|S_u|)$ time.

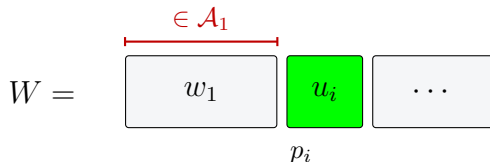
Decompose(u, S_u)

First, we mark each occurrence u_i satisfying $W[..p_i] \in L(\mathcal{A}_1)$, in total $O(|S_u|)$ time ✓

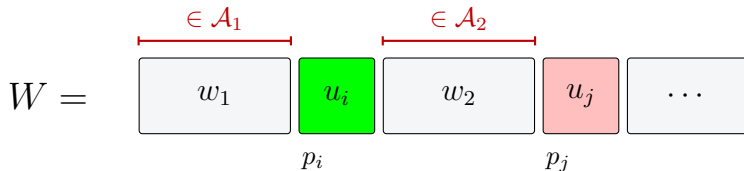


Decompose(u, S_u)

First, we mark each occurrence u_i satisfying $W[..p_i] \in L(\mathcal{A}_1)$, in total $O(|S_u|)$ time ✓

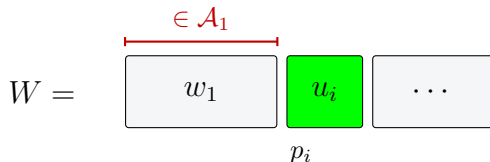


Next, using the marked occurrences, we find a second occurrence u_j such that the middle block belongs to $L(\mathcal{A}_2)$.

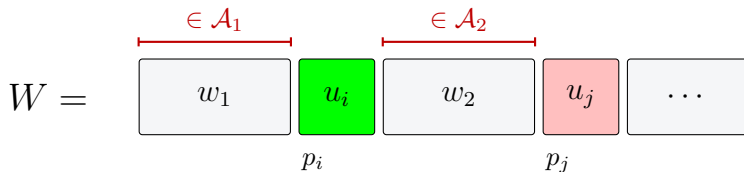


Decompose(u, S_u)

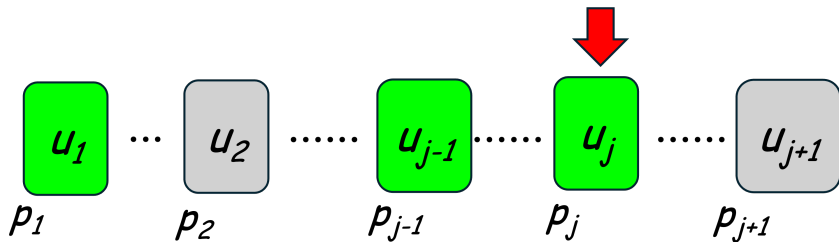
First, we mark each occurrence u_i satisfying $W[..p_i] \in L(\mathcal{A}_1)$, in total $O(|S_u|)$ time ✓



Next, using the marked occurrences, we find a second occurrence u_j such that the middle block belongs to $L(\mathcal{A}_2)$. This can be done in $\mathbf{O}(|S_u|)$ time.

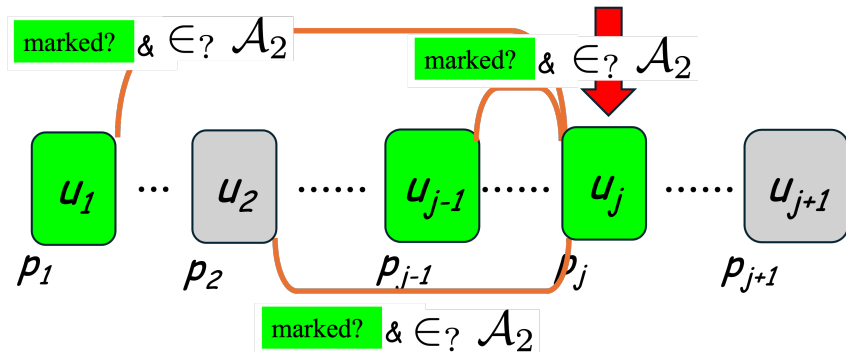


Step 2 (bad): Straightforward *but Slow*



- Now fix $p_j \in S_u$.

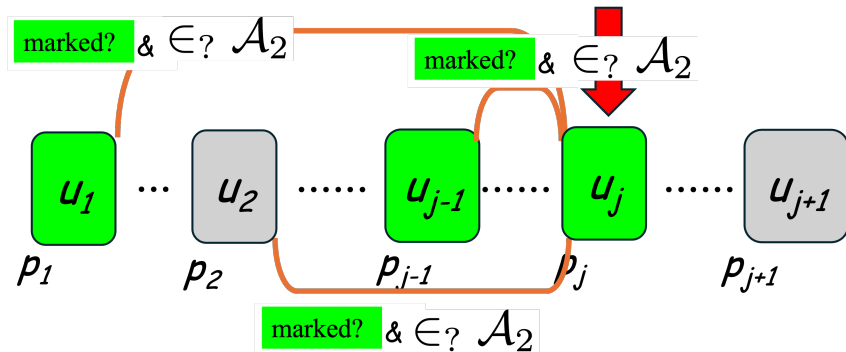
Step 2 (bad): Straightforward *but Slow*



- Now fix $p_j \in S_u$.
- For every $i < j$ such that u_i is **marked**, check whether

$W[p_i + |u| .. p_j) \in L(\mathcal{A}_2)$ in $O(1)$ time.

Step 2 (bad): Straightforward *but Slow*

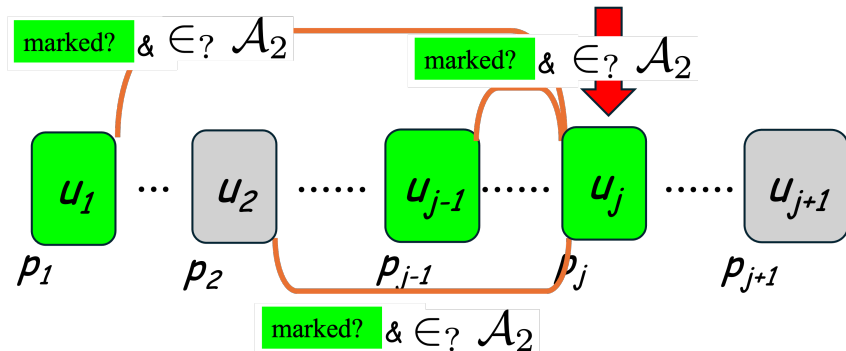


- Now fix $p_j \in S_u$.
- For every $i < j$ such that u_i is **marked**, check whether

$$W[p_i + |u| .. p_j) \in L(\mathcal{A}_2) \text{ in } O(1) \text{ time.}$$

$\Rightarrow O(|S_u|)$ time for this fixed u_j .

Step 2 (bad): Straightforward *but Slow*

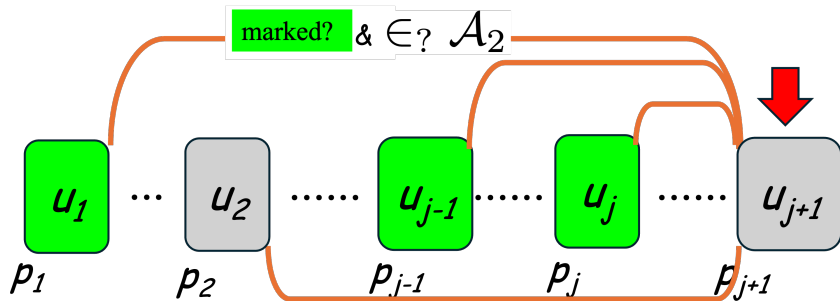


- Now fix $p_j \in S_u$.
- For every $i < j$ such that u_i is **marked**, check whether

$$W[p_i + |u| .. p_j) \in L(\mathcal{A}_2) \text{ in } O(1) \text{ time.}$$

$\Rightarrow O(|S_u|)$ time for this fixed u_j . Hence, $O(|S_u|^2)$ overall.

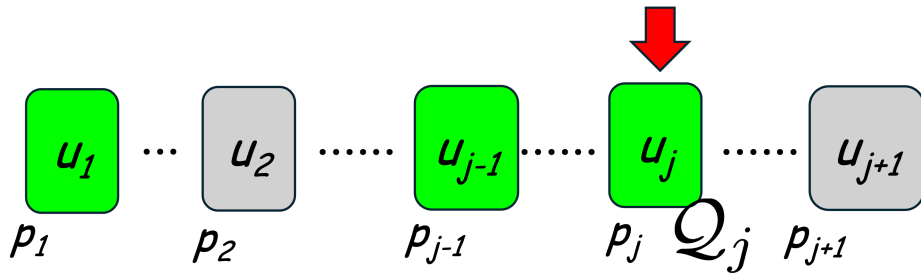
Step 2 (bad): Straightforward but Slow



Why is this approach *inefficient*?

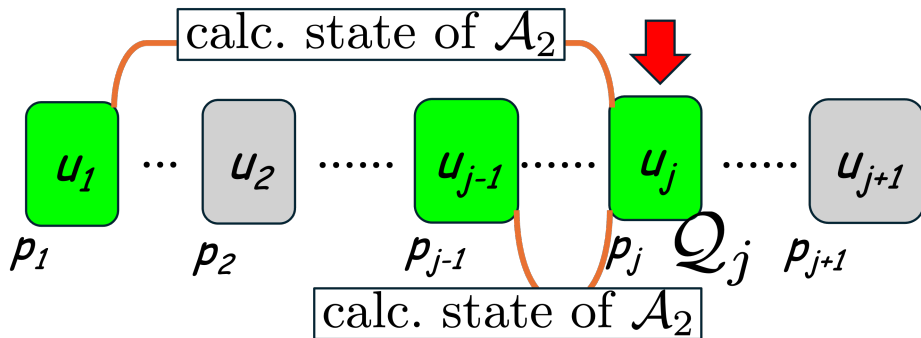
- When moving from u_j to u_{j+1} ,
- we discard the checks done for u_j , and
- recompute similar checks $W[p_i + |u| .. p_{j+1}) \in L(\mathcal{A}_2)$ for all $i < j + 1$.

Step 2 (good): Automaton-State View



For each u_j , we maintain a state set $\mathcal{Q}_j \subseteq \mathcal{Q}_{A_2}$ s.t.:

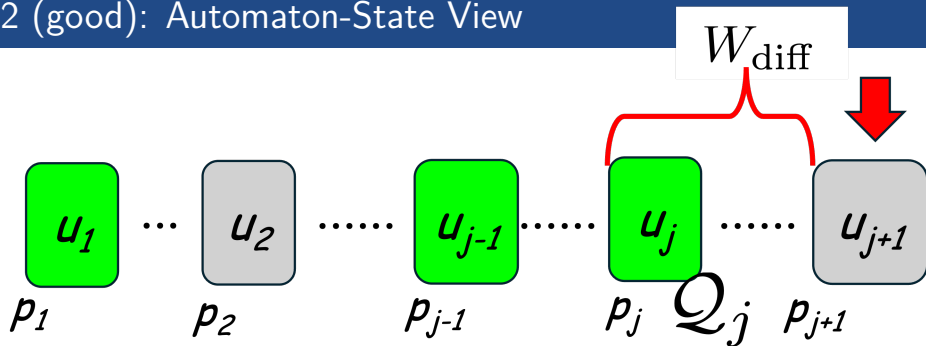
Step 2 (good): Automaton-State View



For each u_j , we maintain a state set $\mathcal{Q}_j \subseteq Q_{\mathcal{A}_2}$ s.t.:

- $q \in \mathcal{Q}_j$ if and only if
- $\exists i < j$. u_i and $\mathcal{A}_2(q^{\text{init}}, W[p_i + |u| .. p_j]) = q$

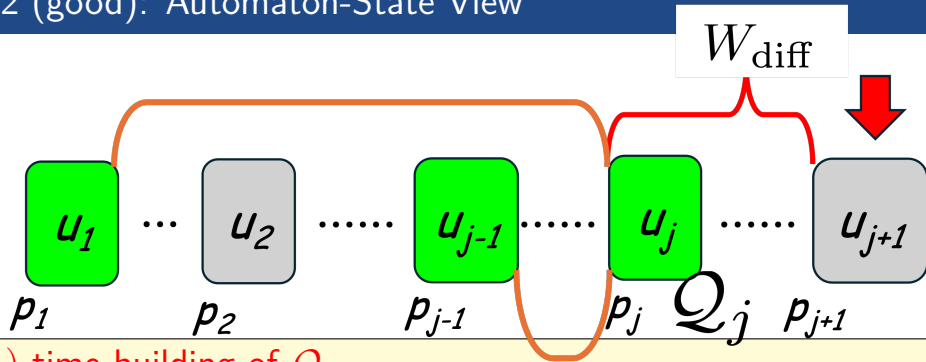
Step 2 (good): Automaton-State View



$O(1)$ time building of Q_{j+1}

- Let $W_{\text{diff}} := W[p_j .. p_{j+1})$.

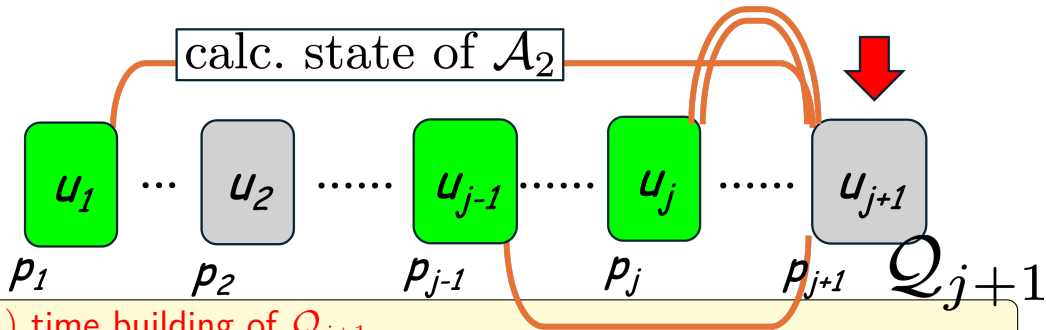
Step 2 (good): Automaton-State View



$O(1)$ time building of Q_{j+1}

- Let $W_{\text{diff}} := W[p_j .. p_{j+1}]$.
- Compute $\mathcal{A}_2(Q_j, W_{\text{diff}}) := \{\mathcal{A}_2(q, W_{\text{diff}}) : q \in Q_j\}$ in $O(|Q_{\mathcal{A}_2}|)$ time.

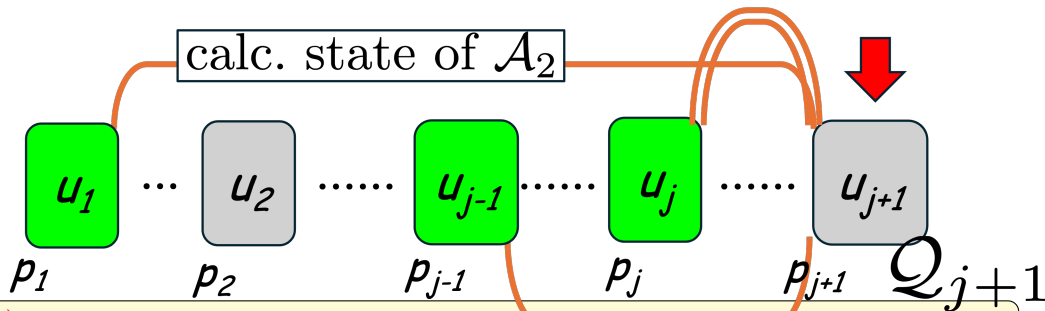
Step 2 (good): Automaton-State View



$O(1)$ time building of $\mathcal{Q}_{j+1}$

- Let $W_{\text{diff}} := W[p_j .. p_{j+1}]$.
- Compute $\mathcal{A}_2(\mathcal{Q}_j, W_{\text{diff}}) := \{\mathcal{A}_2(q, W_{\text{diff}}) : q \in \mathcal{Q}_j\}$ in $O(|\mathcal{Q}_{\mathcal{A}_2}|)$ time.
- Then, $\mathcal{Q}_{j+1} = \mathcal{A}_2(\mathcal{Q}_j, W_{\text{diff}})$ holds! (by treating u_j if it is marked)

Step 2 (good): Automaton-State View



$O(1)$ time building of $\mathcal{Q}_{j+1}$

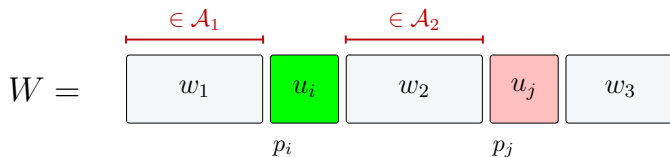
- Let $W_{\text{diff}} := W[p_j .. p_{j+1}]$.
- Compute $\mathcal{A}_2(\mathcal{Q}_j, W_{\text{diff}}) := \{\mathcal{A}_2(q, W_{\text{diff}}) : q \in \mathcal{Q}_j\}$ in $O(|\mathcal{Q}_{\mathcal{A}_2}|)$ time.
- Then, $\mathcal{Q}_{j+1} = \mathcal{A}_2(\mathcal{Q}_j, W_{\text{diff}})$ holds! (by treating u_j if it is marked)

$\Rightarrow$ We can compute $\mathcal{Q}_{j+1}$ in $O(|\mathcal{Q}_{\mathcal{A}_2}|) = O(1)$ time ($\because \mathcal{A}_2$ is a fixed object).

And check if there is an accepting state in $\mathcal{Q}_{j+1}$.

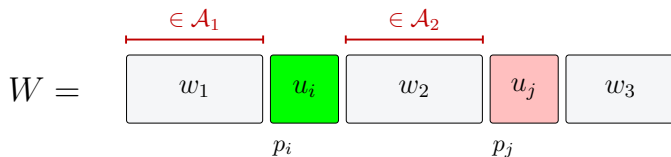
Final Step of Decompose(u, S_u)

We marked *all* good u_j 's satisfying the following in $O(|S_u|)$ time ✓:

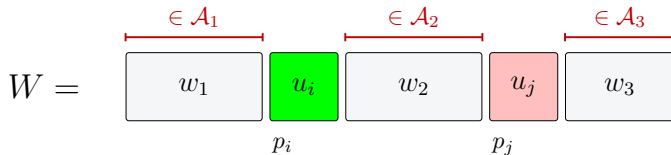


Final Step of Decompose(u, S_u)

We marked *all* good u_j 's satisfying the following in $O(|S_u|)$ time ✓:

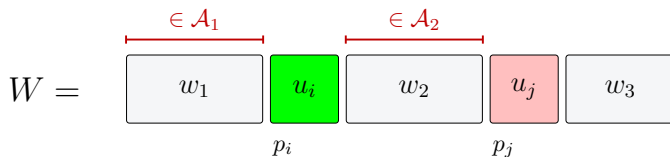


So, we finally search u_j satisfying the following *suffix* condition in $O(|S_u|)$ time:

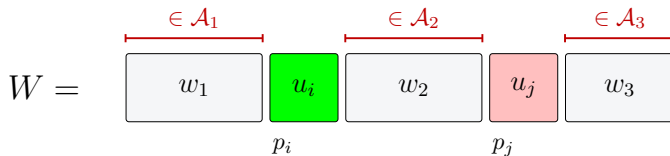


Final Step of Decompose(u, S_u)

We marked *all* good u_j 's satisfying the following in $O(|S_u|)$ time ✓:



So, we finally search u_j satisfying the following *suffix* condition in $O(|S_u|)$ time:



Summary

For fixed u , Decompose(u, S_u) runs in $O(|S_u|)$ time.

Main Procedure

```
for all  $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$  do  
   $\triangleright S_u = [p_1, p_2, \dots, p_r]$  is the sorted occurrence list of  $u$   
  if  $u \notin L(\mathcal{B})$  then  $\triangleright O(1)$  time check by using the data structure for  $\mathcal{B}$   
    continue  
  if  $\text{Decompose}(u, S_u)$  then  
     $\triangleright$  check if there is a decomposition  $W = w_1 u w_2 u w_3$  in  $O(|S_u|)$  time  
  return ACCEPT  
return REJECT
```

Since $\text{Decompose}(u, S_u)$ runs in $O(|S_u|)$ time,

Main Procedure

```
for all  $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$  do  
   $\triangleright S_u = [p_1, p_2, \dots, p_r]$  is the sorted occurrence list of  $u$   
  if  $u \notin L(\mathcal{B})$  then  $\triangleright O(1)$  time check by using the data structure for  $\mathcal{B}$   
    continue  
  if  $\text{Decompose}(u, S_u)$  then  
     $\triangleright$  check if there is a decomposition  $W = w_1 u w_2 u w_3$  in  $O(|S_u|)$  time  
  return ACCEPT  
return REJECT
```

Since $\text{Decompose}(u, S_u)$ runs in $O(|S_u|)$ time,
the total time complexity is $\sum_{\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)} |S_u|$

Main Procedure

```
for all  $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$  do  
   $\triangleright S_u = [p_1, p_2, \dots, p_r]$  is the sorted occurrence list of  $u$   
  if  $u \notin L(\mathcal{B})$  then  $\triangleright O(1)$  time check by using the data structure for  $\mathcal{B}$   
    continue  
  if  $\text{Decompose}(u, S_u)$  then  
     $\triangleright$  check if there is a decomposition  $W = w_1 u w_2 u w_3$  in  $O(|S_u|)$  time  
  return ACCEPT  
return REJECT
```

Since $\text{Decompose}(u, S_u)$ runs in $O(|S_u|)$ time,
the total time complexity is $\sum_{\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)} |S_u| = \Theta(|W|^2)$.

Main Procedure

```
for all  $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$  do  
   $\triangleright S_u = [p_1, p_2, \dots, p_r]$  is the sorted occurrence list of  $u$   
  if  $u \notin L(\mathcal{B})$  then  $\triangleright O(1)$  time check by using the data structure for  $\mathcal{B}$   
    continue  
  if  $\text{Decompose}(u, S_u)$  then  
     $\triangleright$  check if there is a decomposition  $W = w_1 u w_2 u w_3$  in  $O(|S_u|)$  time  
    return ACCEPT  
return REJECT
```

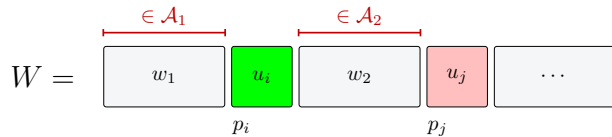
Since $\text{Decompose}(u, S_u)$ runs in $O(|S_u|)$ time,
the total time complexity is $\sum_{\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)} |S_u| = \Theta(|W|^2)$.

Conclusion

The time complexity of our algorithm is $O(|W|^2)$ time.

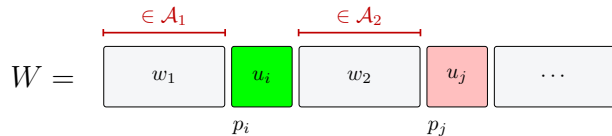
Generalization to $K = 4$

We first proceed to the following situation through the process saw above:

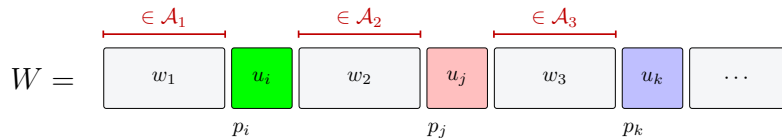


Generalization to $K = 4$

We first proceed to the following situation through the process saw above:

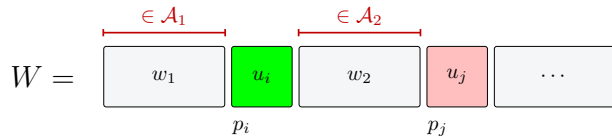


Then, we **again** apply the Decompose (using the DFA $\mathcal{A}_3$) to obtain the following situation:

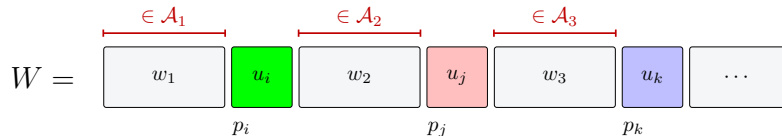


Generalization to $K = 4$

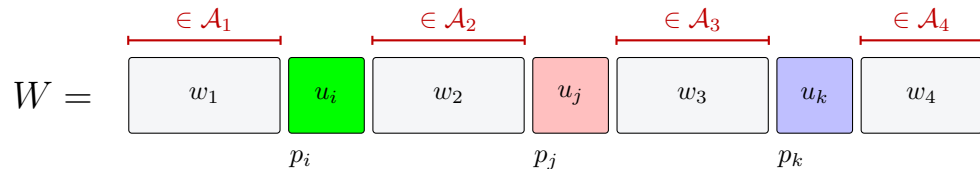
We first proceed to the following situation through the process saw above:



Then, we **again** apply the Decompose (using the DFA $\mathcal{A}_3$) to obtain the following situation:



Finally, we check the suffixes from candidates u_k



Conclusion

For $P_K = \alpha_1 \beta \alpha_2 \beta \alpha_3 \cdots \beta \alpha_K$, we give an $O_K(|W|^2)$ -time algorithm.

Conclusion

For $P_K = \alpha_1 \beta \alpha_2 \beta \alpha_3 \cdots \beta \alpha_K$, we give an $O_K(|W|^2)$ -time algorithm.

Question: Can we achieve a sub-quadratic time algorithm?

Conclusion

For $P_K = \alpha_1 \beta \alpha_2 \beta \alpha_3 \cdots \beta \alpha_K$, we give an $O_K(|W|^2)$ -time algorithm.

Question: Can we achieve a sub-quadratic time algorithm?

Answer: Yes at least for $K = 3$, $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$.

We recently show an $O(|W| \cdot \log^2(|W|))$ -time algorithm.

It will appear in ICALP 2026 (by Me & Soh Kumabe; <https://arxiv.org/abs/2605.07289>).

Conclusion

For $P_K = \alpha_1 \beta \alpha_2 \beta \alpha_3 \cdots \beta \alpha_K$, we give an $O_K(|W|^2)$ -time algorithm.

Question: Can we achieve a sub-quadratic time algorithm?

Answer: **Yes** at least for $K = 3$, $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$.

We recently show an $O(|W| \cdot \log^2(|W|))$ -time algorithm.

It will appear in ICALP 2026 (by Me & Soh Kumabe; <https://arxiv.org/abs/2605.07289>).

Question: Is there any other an interesting **single variable** examples of pattern expressions or **regular expressions with backreferences**?

Answer: Consider a regular expression with a single **reusing backreference** variable:

$$\text{REP} = ((\Sigma^+)_1 \setminus 1)^+$$

Conclusion

For $P_K = \alpha_1 \beta \alpha_2 \beta \alpha_3 \cdots \beta \alpha_K$, we give an $O_K(|W|^2)$ -time algorithm.

Question: Can we achieve a sub-quadratic time algorithm?

Answer: **Yes** at least for $K = 3$, $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$.

We recently show an $O(|W| \cdot \log^2(|W|))$ -time algorithm.

It will appear in ICALP 2026 (by Me & Soh Kumabe; <https://arxiv.org/abs/2605.07289>).

Question: Is there any other an interesting **single variable** examples of pattern expressions or **regular expressions with backreferences**?

Answer: Consider a regular expression with a single **reusing backreference** variable:

$$\text{REP} = ((\Sigma^+)_1 \setminus 1)^+ = (\alpha_1 \alpha_1) \cup (\alpha_1 \alpha_1 \alpha_2 \alpha_2) \cup (\alpha_1 \alpha_1 \alpha_2 \alpha_2 \alpha_3 \alpha_3) \cup \cdots .$$

Conclusion

For $P_K = \alpha_1 \beta \alpha_2 \beta \alpha_3 \cdots \beta \alpha_K$, we give an $O_K(|W|^2)$ -time algorithm.

Question: Can we achieve a sub-quadratic time algorithm?

Answer: **Yes** at least for $K = 3$, $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$.

We recently show an $O(|W| \cdot \log^2(|W|))$ -time algorithm.

It will appear in ICALP 2026 (by Me & Soh Kumabe; <https://arxiv.org/abs/2605.07289>).

Question: Is there any other an interesting **single variable** examples of pattern expressions or **regular expressions with backreferences**?

Answer: Consider a regular expression with a single **reusing backreference** variable:

$$\text{REP} = ((\Sigma^+)_1 \setminus 1)^+ = (\alpha_1 \alpha_1) \cup (\alpha_1 \alpha_1 \alpha_2 \alpha_2) \cup (\alpha_1 \alpha_1 \alpha_2 \alpha_2 \alpha_3 \alpha_3) \cup \cdots .$$

Surprisingly, we can decide if $W \in L(\text{REP})$ in **linear** $O(|W|)$ **time**.

Factorizing a String into Squares in Linear Time @ CPM 2016

Yoshiaki Matsuoka¹, Shunsuke Inenaga², Hideo Bannai³,
Masayuki Takeda⁴, and Florin Manea⁵

Conclusion

For $P_K = \alpha_1 \beta \alpha_2 \beta \alpha_3 \cdots \beta \alpha_K$, we give an $O_K(|W|^2)$ -time algorithm.

Question: Can we achieve a sub-quadratic time algorithm?

Answer: **Yes** at least for $K = 3$, $P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3$.

We recently show an $O(|W| \cdot \log^2(|W|))$ -time algorithm.

It will appear in ICALP 2026 (by Me & Soh Kumabe; <https://arxiv.org/abs/2605.07289>).

Question: Is there any other an interesting **single variable** examples of pattern expressions or **regular expressions with backreferences**?

Answer: Consider a regular expression with a single **reusing backreference** variable:

$$\text{REP} = ((\Sigma^+)_1 \setminus 1)^+ = (\alpha_1 \alpha_1) \cup (\alpha_1 \alpha_1 \alpha_2 \alpha_2) \cup (\alpha_1 \alpha_1 \alpha_2 \alpha_2 \alpha_3 \alpha_3) \cup \cdots .$$

Surprisingly, we can decide if $W \in L(\text{REP})$ in **linear** $O(|W|)$ **time**.

Factorizing a String into Squares in Linear Time @ CPM 2016

Yoshiaki Matsuoka¹, Shunsuke Inenaga², Hideo Bannai³,
Masayuki Takeda⁴, and Florin Manea⁵



I am happy if you are interested in the REGEX world with a few (repeating/reusing) variables.

Bonus slide

Example: Text Search via Regular-Typed Patterns

Let $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{z}, \#\}$ and let α, β, γ be variables. Consider the pattern

$$P = \beta \# \alpha \beta \gamma.$$

Example: Text Search via Regular-Typed Patterns

Let $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{z}, \#\}$ and let α, β, γ be variables. Consider the pattern

$$P = \beta \# \alpha \beta \gamma.$$

To exclude the separator $\#$ in the variables, we can use the following regular type:

$$\text{WORD} = C(\alpha) = C(\beta) = C(\gamma) = (\Sigma \setminus \{\#\})^*.$$

Example: Text Search via Regular-Typed Patterns

Let $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{z}, \#\}$ and let α, β, γ be variables. Consider the pattern

$$P = \beta \# \alpha \beta \gamma.$$

To exclude the separator $\#$ in the variables, we can use the following regular type:

$$\text{WORD} = C(\alpha) = C(\beta) = C(\gamma) = (\Sigma \setminus \{\#\})^*.$$

$$W \in L_C(P) \iff W = \boxed{\overset{\textcolor{red}{\vdash} \in \text{WORD} \textcolor{red}{\dashv}}{w_\beta}} \# \boxed{\overset{\textcolor{red}{\vdash} \in \text{WORD} \textcolor{red}{\dashv}}{w_\alpha}} \boxed{\overset{\textcolor{red}{\vdash} \in \text{WORD} \textcolor{red}{\dashv}}{w_\beta}} \boxed{\overset{\textcolor{red}{\vdash} \in \text{WORD} \textcolor{red}{\dashv}}{w_\gamma}}$$

Thus, P searches for the word w_β before $\#$ in the body part after $\#$.

Example: Text Search via Regular-Typed Patterns

Let $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{z}, \#\}$ and let α, β, γ be variables. Consider the pattern

$$P = \beta \# \alpha \beta \gamma.$$

To exclude the separator $\#$ in the variables, we can use the following regular type:

$$\text{WORD} = C(\alpha) = C(\beta) = C(\gamma) = (\Sigma \setminus \{\#\})^*.$$

$$W \in L_C(P) \iff W = \boxed{\overset{\vdash \in \text{WORD} \dashv}{w_\beta}} \# \boxed{\overset{\vdash \in \text{WORD} \dashv}{w_\alpha}} \boxed{\overset{\vdash \in \text{WORD} \dashv}{w_\beta}} \boxed{\overset{\vdash \in \text{WORD} \dashv}{w_\gamma}}$$

Thus, P searches for the word w_β before $\#$ in the body part after $\#$.

Example

$\mathbf{aba} \# \mathbf{c} \mathbf{aba} \mathbf{d} \in L_C(P), \quad \beta \mapsto \mathbf{aba}, \quad \alpha \mapsto \mathbf{c}, \quad \gamma \mapsto \mathbf{d},$

$\mathbf{aba} \# \mathbf{c} \mathbf{abz} \mathbf{d} \notin L_C(P).$